



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 5.1

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
1.1. Contexto.....	3
1.2. Objetivos del Sprint 5.1.....	3
1.3. Justificación.....	3
2. Metodología.....	4
2.1. Respuesta a HU-6: Adquisición de Datos en Tiempo Real.....	4
2.2. Configuración del Topic de Kafka.....	4
2.3. Adquisición de Datos desde Binance.....	4
2.4. Publicación de Mensajes en Kafka.....	5
3. Resultados.....	6
3.1. Descripción de los Resultados Obtenidos.....	6
3.2. Estructura del Mensaje Publicado y Validación.....	6
4. Conclusión.....	8
4.1. Resumen del Proceso.....	8
4.2. Principales Logros.....	8

1. Introducción

1.1. Contexto

Esta primera parte del Sprint 5 representa el inicio de la parte de procesamiento en tiempo real del proyecto TradeData. En los cuatro sprints anteriores hemos construido una infraestructura progresiva y sólida: en el Sprint 1 implementamos la ingesta y almacenamiento de datos históricos de nuestra criptomoneda SOLUSD bajo la capa Bronze; en el Sprint 2 establecimos el gobierno del dato mediante AWS Glue Data Catalog y sus Crawlers asociados; en el Sprint 3 completamos el pipeline Bronze → Silver → Gold calculando indicadores técnicos como el SMA 200, el EMA 50, el RSI y el MACD mediante Apache Spark sobre AWS Glue; y en el Sprint 4 implementamos la capa de visualización sobre Amazon QuickSight, cerrando el ciclo completo de los datos históricos.

Con todo esto correctamente asentado, en el Sprint 5.1 damos el salto al procesamiento en tiempo real, incorporando Apache Kafka como plataforma de mensajería distribuida para la ingesta de datos de mercado directamente desde Binance, que tiene como finalidad almacenar la información de cotizaciones de SOLUSD en tiempo real mediante la publicación de mensajes en un topic de Kafka.

1.2. Objetivos del Sprint 5.1

Los objetivos concretos del Sprint 5.1, tal y como se recogen en la HU-6, son los siguientes:

- Crear el topic de Kafka correspondiente al grupo, con una partición y factor de replicación 1.
- Implementar la adquisición de datos en tiempo real desde Binance para la criptomoneda asignada al grupo.
- Publicar mensajes en el topic de Kafka únicamente cuando la vela esté cerrada, con la clave igual al símbolo de la criptomoneda y el valor en formato JSON, incluyendo los campos *symbol*, *@timestamp*, *close* y *volume*.
- Asegurar que el campo *@timestamp* se obtiene a partir de los datos proporcionados por Binance, expresado en formato UTC sin zona horaria, y que el timestamp en milisegundos también proviene de Binance.

1.3. Justificación

El paso de datos históricos a datos en tiempo real responde a una necesidad clave en el análisis de mercados financieros: no basta con conocer el comportamiento pasado de un activo, sino que es esencial contar con información actualizada de forma continua para tomar decisiones a tiempo. En este contexto, el procesamiento *streaming* cobra sentido por su baja latencia y por trabajar con datos generados continuamente en forma de flujos, a diferencia del procesamiento *batch*, que opera sobre datos estáticos y añade más latencia.

Apache Kafka es la herramienta elegida para la capa de almacenamiento intermedio (*Data Storage*) en esta arquitectura de procesamiento *streaming*, tal como recoge la teoría de la asignatura. Su escalabilidad, durabilidad, tolerancia a fallos y baja latencia lo convierten en una solución estándar para gestionar flujos de datos en tiempo real.

2. Metodología

2.1. Respuesta a HU-6: Adquisición de Datos en Tiempo Real

La Historia de Usuario 6 establece que el equipo debe implementar un pipeline de adquisición de datos en tiempo real que conecte la API de Binance con un topic de Kafka. Este pipeline se articula en dos componentes principales: por un lado, un proceso que se suscribe al websocket de Binance y recibe velas de forma continua; por otro, un producer de Kafka que publica cada vela cerrada como un mensaje en el topic correspondiente.

Para abordar esta historia se han utilizado las plantillas de código proporcionadas por el equipo docente: el script `binance_real_time.py` como base para la adquisición de datos desde Binance, y el script `kafka_simple_producer.py` como referencia para la publicación de mensajes en Kafka.

2.2. Configuración del Topic de Kafka

Según las instrucciones de la HU-6, cada grupo debe crear un topic de Kafka con una única partición y un factor de replicación igual a 1. En nuestro caso el topic que hemos creado es `imat3a_SOL_BigDaddyks`, que es también el identificador del consumer group asociado.

La conexión con el clúster de Kafka la hemos configurado a través del servidor de bootstrap proporcionado por el equipo docente, empleando el protocolo `SASL_PLAINTEXT` con mecanismo `PLAIN` para la autenticación. Los parámetros de conexión son los siguientes:

- Bootstrap server: `51.49.235.244:9092`
- Protocolo de seguridad: `SASL_PLAINTEXT`
- Mecanismo SASL: `PLAIN`
- Usuario: `kafka_client`

2.3. Adquisición de Datos desde Binance

La adquisición de datos en tiempo real la hemos implementado apoyándonos en la librería `python-binance`, que proporciona la clase `ThreadedWebsocketManager`. Esta clase permite definir una función de callback que se invoca cada vez que Binance emite un nuevo mensaje de tipo `kline` (vela).

Como punto de partida hemos adaptado el script `binance_real_time.py` a nuestro par `SOLUSDT` con intervalo de un minuto. Cada mensaje recibido contiene un objeto `k` con toda la información de la vela en curso, incluyendo el precio de cierre (`c`), el volumen (`v`), el símbolo (`s`), el timestamp de apertura (`t`) y el timestamp de cierre (`T`), así como el campo `x` que indica si la vela ha terminado.

La condición clave que impone la HU-6 es que solo debemos publicar mensajes en Kafka cuando la vela esté cerrada. Por ello, dentro de la función de callback evaluamos el campo `k['x']`, que toma el valor `True` únicamente cuando la vela ha concluido, y solo en ese caso construimos el mensaje y lo enviamos al topic.

El campo `@timestamp` lo obtenemos a partir del timestamp de cierre de la vela que proporciona Binance (campo `T` del objeto `k`), expresado originalmente en milisegundos. Para

cumplir con el formato requerido por la HU-6 (UTC, sin zona horaria, con el formato `YYYY-MM-DDTHH:MM:SSZ`), convertimos dicho valor a un objeto `datetime` en UTC y lo formateamos como cadena de texto. El timestamp en milisegundos que incluimos como campo en el mensaje es también el valor original de Binance, sin transformación.

2.4. Publicación de Mensajes en Kafka

Para la publicación de mensajes hemos utilizado la clase `KafkaProducer` de la librería `kafka-python`. La clave del mensaje (*key*) la hemos establecido como la constante `SYMBOL`, con valor `SOLUSD`, mientras que el valor (*value*) es un objeto JSON construido a partir de los campos del objeto `k` que devuelve Binance, con la siguiente estructura:

```
{ "symbol": "SOLBTC", "@timestamp": "2026-03-09T11:21:00Z", "close": 134.52, "volume": 8523.41 }
```

La conversión del timestamp de milisegundos a formato UTC la realizamos mediante la función `ms_to_utc_z`, que toma el campo `T` del objeto `k`, lo divide entre 1000 para obtener segundos epoch, lo convierte a un objeto `datetime` en UTC y lo formatea como cadena con el patrón `YYYY-MM-DDTHH:MM:SSZ`. El producer no se cierra entre mensajes, sino que permanece activo durante toda la sesión, acumulando el envío de cada vela cerrada sin overhead de reconexión.

3. Resultados

3.1. Descripción de los Resultados Obtenidos

Tras la implementación descrita en la sección de metodología, el pipeline de adquisición y publicación en tiempo real funciona de la siguiente manera: el proceso se conecta al websocket de Binance y comienza a recibir actualizaciones de la vela activa de *SOLUSDT* en tiempo real, con una frecuencia de un minuto. Cada minuto, al cierre de la vela, generamos un mensaje que publicamos en el topic *imat3a_SOL_BigDaddyks* del clúster de Kafka.

La publicación se produce exclusivamente sobre la partición 0 del topic, en coherencia con la configuración de una única partición establecida en la HU-6. Cada mensaje queda identificado de forma unívoca por su offset dentro de la partición, lo que garantiza la trazabilidad de cada vela recibida y la posibilidad de recuperar la secuencia de mensajes en caso de fallo, tal y como contempla el mecanismo de *__consumer_offsets* de Kafka.

```
[ec2-user@ip-172-31-24-254 ~]$ python3 solusd_real_time.py
Pulsa ENTER para salir
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:24:59Z', 'close': '0.00123010', 'volume': '1.40600000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:25:59Z', 'close': '0.00123050', 'volume': '1.25400000'}
Mensaje a enviar: SOLUSD {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:26:59Z', 'close': '0.00123070', 'volume': '0.11100000'}
```

Figura 1. Producer publicando el mensaje por el topic

Para verificar la correcta recepción de los mensajes hemos utilizado un script análogo al *kafka_simple_consumer.py* proporcionado, configurado para leer del topic *imat3a_SOL_BigDaddyks* y la partición 0. La lectura nos muestra para cada mensaje la clave (*SOLUSDT*), el valor (objeto JSON con los campos requeridos), el offset y el timestamp del mensaje.

```
[ec2-user@ip-172-31-24-254 ~]$ python3 kafka_simple_consumer.py
TopicPartition(topic='imat3a_SOL_BigDaddyks', partition=0)
key: SOLUSD
value: {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:24:59Z', 'close': '0.00123010', 'volume': '1.40600000'}
offset: 1
timestamp: 1773339900278
TopicPartition(topic='imat3a_SOL_BigDaddyks', partition=0)
key: SOLUSD
value: {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:25:59Z', 'close': '0.00123050', 'volume': '1.25400000'}
offset: 2
timestamp: 1773339960226
TopicPartition(topic='imat3a_SOL_BigDaddyks', partition=0)
key: SOLUSD
value: {'symbol': 'SOLBTC', '@timestamp': '2026-03-12T18:26:59Z', 'close': '0.00123070', 'volume': '0.11100000'}
offset: 3
timestamp: 1773340020275
```

Figura 2. Consumer recibiendo el mensaje por el topic

3.2. Estructura del Mensaje Publicado y Validación

A modo de ejemplo, un mensaje publicado en el topic *imat3a_SOL_BigDaddyks* tras el cierre de una vela tiene la siguiente estructura:

Clave (*key*): *SOLUSD*

Valor (*value*): { "symbol": "SOLBTC", "@timestamp": "2026-03-09T11:21:00Z", "close": 134.52, "volume": 8523.41 }

Hemos verificado la correcta recepción ejecutando el consumer, que corre en un bucle continuo (*while True*) realizando polls periódicos sobre la partición 0 del topic. Para cada mensaje recibido se imprime la clave (*SOLUSD*), el valor (objeto JSON con los campos requeridos), el offset y el timestamp. Hemos comprobado que:

- El campo *@timestamp* corresponde al timestamp de cierre de la vela obtenido directamente de Binance, convertido a formato UTC sin zona horaria mediante la función *ms_to_utc_z*.
 - El mensaje se publica únicamente al cierre de la vela, condicionado por el campo *k['x']* igual a True en el script *solusd_real_time.py*.
 - La serialización del valor en JSON es compatible con la deserialización implementada en el consumer, que emplea *json.loads* sobre la cadena decodificada en UTF-8.
-

4. Conclusión

4.1. Resumen del Proceso

En esta parte del sprint hemos implementado el pipeline de adquisición y publicación de datos de *SOLUSD* en tiempo real, respondiendo a los requerimientos de la HU-6. El proceso parte de la conexión con el websocket de Binance mediante la librería *python-binance*, recibe las actualizaciones de velas en tiempo real y, al detectar el cierre de cada vela, construye un mensaje JSON con los campos *symbol*, *@timestamp*, *close* y *volume*, y lo publica en el topic *imat3a_SOL_BigDaddyks* empleando la librería *kafka-python*.

La implementación respeta todas las restricciones técnicas indicadas en la historia de usuario: el *@timestamp* se obtiene a partir del dato de Binance y se expresa en UTC sin zona horaria, el timestamp en milisegundos también proviene de Binance, y la publicación se condiciona estrictamente al cierre de la vela.

4.2. Principales Logros

Los logros principales del Sprint 5.1 incluyen:

1. Pipeline de adquisición en tiempo real implementado: Hemos conectado el websocket de Binance con el topic de Kafka *imat3a_SOL_BigDaddyks*, publicando mensajes de *SOLUSD* cada minuto al cierre de cada vela, con todos los campos y restricciones que exige la HU-6.
2. Incorporación del procesamiento streaming a la arquitectura: Hemos dado el salto de los datos históricos estáticos al tiempo real, completando así la capa de Data Storage del flujo de procesamiento streaming que la teoría de la asignatura identifica como solución al problema de la velocidad en entornos Big Data.
3. Comprensión práctica de Apache Kafka: Hemos consolidado el uso de Kafka como bus de mensajes distribuido, trabajando con sus conceptos fundamentales (topics, particiones, offsets, producers y brokers) y configurando la autenticación SASL en un clúster real.
4. Base para el análisis en tiempo real: La correcta ingesta de mensajes en Kafka sienta las bases para que, en sprints futuros, un consumer pueda leer del topic *imat3a_SOL_BigDaddyks* y aplicar lógica de procesamiento sobre el flujo de datos, cerrando el ciclo completo de la arquitectura de streaming.